

Whiley: a Platform for Research in Software Verification

David J. Pearce

*School of Engineering and Computer Science
Victoria University of Wellington*

`http://whiley.org`

Verifying Compilers

- Tony Hoare proposed the development of a verifying compiler as **grand challenge** for Computer Science (2003)

“A verifying compiler uses automated mathematical and logical reasoning methods to check the correctness of the programs that it compiles”

- Many **impressive** efforts in this space:

SPARK/Ada, Eiffel, ESC/Modula3, ESC/Java, JML/OpenJML, Why/Krakatoa, Spec#, Dafny, VeriFast, Frama-C

- But, still **lots** of research to be done!



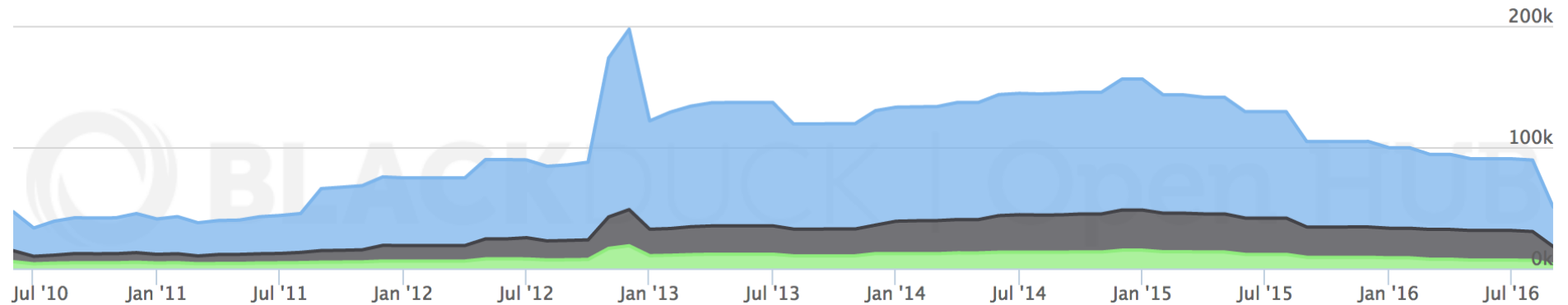
Whiley

Overview: What is Whiley?

```
function max(int x, int y) -> (int z)
// result must be one of the arguments
ensures x == z || y == z
// result must be greater-or-equal than arguments
ensures x <= z && y <= z :
    . . .
```

- A language designed specifically to simplify **verifying software**
- Several trade offs e.g. **performance for verifiability**
 - *Unbounded Arithmetic, value semantics, etc*
- **Goal:** to statically verify functions meet their specifications

History of Whiley



- 2009 — **Initial** version of Whiley released (GPL Licence)
- 2010 — **GitHub** repository and <http://whiley.org> go live
- 2010 — **Version 0.3.0** released (BSD Licence)
- 2016 — **Version 0.4.0** released (approx 60KLOC)

Demo (Part 1)

`abs ()`

`indexOf ()`

`sortedIndexOf ()`



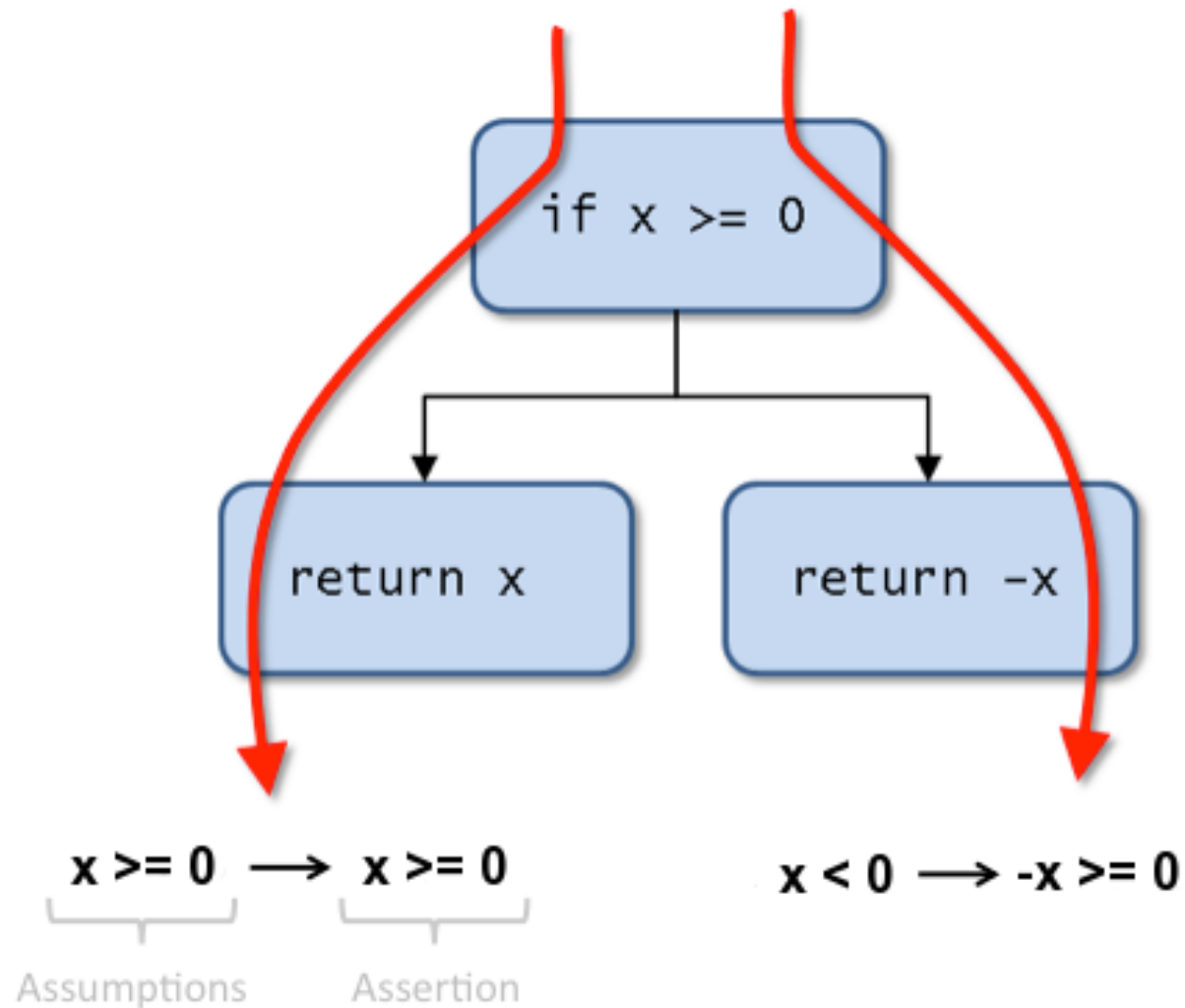
Verification

Verification: Mechanism

```
function abs (int x) ==> (int r)
// return value cannot be negative
ensures r >= 0:
    //
    if x >= 0:
        return x
    else:
        return -x
```

- To verify above function, compiler generates **verification conditions**
- Verification conditions are (roughly) **first-order logic formulas**

Verification: Example



Verification: Assertion Language

- Whiley compiler emits verification conditions in **assertion language**

assert :

```
forall (int x) :  
    x >= 0 ==> x >= 0
```

assert :

```
forall (int x) :  
    x < 0 ==> -x >= 0
```

- Verification conditions from `abs()` example shown above
- In principle, can hook up different **automatic theorem provers**

Demo (Part 2)

`zeroOut ()`

`strlen ()`

Verification: Implicit Assumptions

```
constant NULL is 0
```

```
// A null-terminated character array
```

```
type C_string is (int[] str)
```

```
where some { i in 0 .. |str| | str[i] == NULL }
```

```
// Determine length of null-terminated string
```

```
function strlen(C_string str) -> (int r)
```

```
ensures str[r] == NULL
```

```
ensures all { k in 0 .. r | str[k] != NULL }:
```

```
//
```

- Should above specification be **acceptable** ... ?

Verification: Constrained Types

```
type N is (T x) where e
```

- Above defines **constrained type**
- **Invariant:** for any variable of type N , follows that e always holds
- Constrained types can **simplify** specifications / invariants
- **Example:** *natural numbers*

```
type nat is (int n) where n >= 0
```

Verification: Structural Typing

```
type nat is (int n) where n >= 0
```

```
function cut (int x) -> (nat y) :
```

```
  if x >= 0:
```

```
    return x
```

```
  else:
```

```
    return 0
```

- Variable types in Whiley are **ephemeral** ...

... and determined by what is **known** (not what was declared)

Verification: Flow Typing

```
function indexOf(int[] items, int item) -> (int|null r)
// If integer value returned, must be index of item
ensures r is int ==> items[r] == item
// No element before integer r matches item
ensures r is int ==> all { k in 0..r | items[k] != item }
// If null returned, no matching item
ensures r is null ==> all { k in 0..|items| | items[k] != item }:
    //
    int i = 0
    //
    while i < |items|
    where i >= 0 && i <= |items|
    where all { j in 0..i | items[j] != item }:
        if items[i] == item:
            return i
        i = i + 1
    //
    return null
```

Verification: Break Statements

```
function findNeg(int[] items) -> (int r):  
  //  
  int i = 0  
  //  
  while i < |items|  
  where i >= 0 && i <= |items|:  
    //  
    if items[i] < 0:  
      i = -i           // breaks invariant!  
      break  
    //  
    i = i + 1  
  //  
  return i
```

- Should this be **acceptable** ... ?

Verification: Lack of Predicates

```
type nat is (int n) where n >= 0

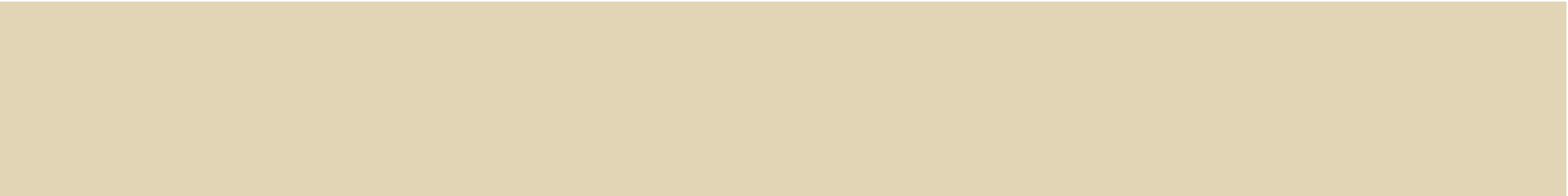
function fill(int[] items, int n) -> (int[] rs)
ensures all { k in 0..|rs| | rs[k] == n }
ensures |items| == |rs|:
  //
  nat i = 0
  int size = |items| //ghost
  //
  while i < |items|
  where |items| == size
  where all { j in 0..i | items[j] == n }:
    items[i] = n
    i = i + 1
  //
  return items
```

- **Repetition** in loop invariants undesirable
- Cannot simply use functions as they are **uninterpreted**

Verification: Lack of Invariant Synthesis

```
function sum(int[] xs) -> (int r) :  
    //  
    int x = 0  
    int i = 0  
    //  
    while i < |xs| :  
        x = x + xs[i]  
        i = i + 1  
    //  
    return x
```

- This example **doesn't verify** — loop invariant is too weak
 - Must strengthen invariant to **where** $i \geq 0$
- This is **frustrating** and would like compiler to do this automatically



Automated Theorem Proving

Theorem Proving: Rewriting

- Whiley's automated theorem prover generated from **rewrite rules**:

// Eliminate conjuncts which are false.

```
reduce And{False, BExpr... xs}:  
    => False
```

// A conjunct containing an expression and its inversion is unsatisfiable.

```
reduce And{Not (BExpr x), BExpr y, BExpr... ys}:  
    => False, if x == y
```

// Distribute conjuncts over disjuncts.

```
reduce And{Or{BExpr... xs}, BExpr... ys}:  
    => let ys = { And(x ++ ys) | x in xs }  
        in Or(ys)
```

- Rules for *logic, linear arithmetic, uninterpreted functions, ...*

Theorem Proving: Search Space Exploration

Observation

Inequality closure and **quantifier instantiation** make the system non-confluent

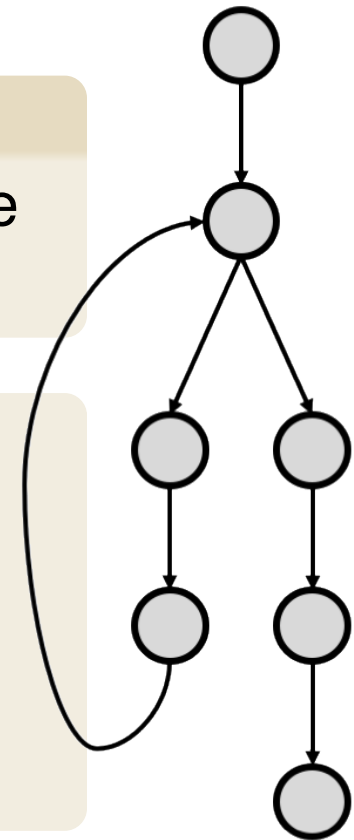
$$1 < x \wedge x < 2$$

$$\implies 1 < x \wedge x < 2 \wedge 1 < 2$$

$$\implies 1 < x \wedge x < 2 \wedge \text{true}$$

$$\implies 1 < x \wedge x < 2$$

(infer)
(reduce)
(reduce)

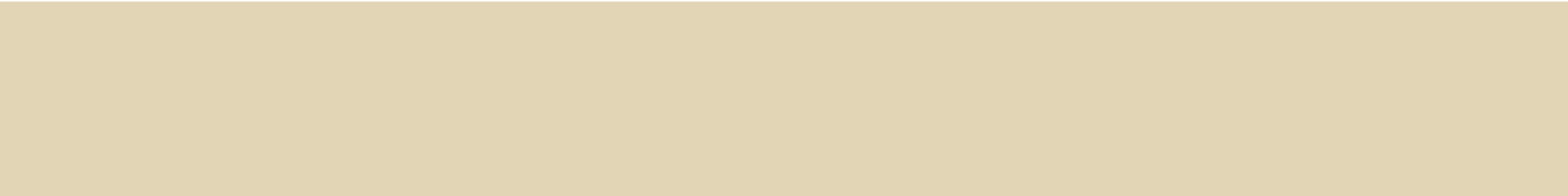


Theorem Proving: Counting Problem

$$\begin{aligned} & \forall x. (x(i) < x(i+1)) \wedge y(0) \geq y(1) \\ \implies & \dots \wedge y(1) < y(1+1) && (infer) \\ \implies & \dots \wedge y(1) < y(2) && (reduce) \\ \implies & \dots \wedge y(1) < y(2) \wedge y(2) < y(2+1) && (infer) \\ \implies & \dots \wedge y(1) < y(2) \wedge y(2) < y(3) && (reduce) \\ & \dots \end{aligned}$$

- Counting problem leads to **infinite loop** within theorem prover
- Can try **bread-first** not **depth-first** exploration
- Even **Z3** seems to have some problems with this kind of thing:

```
(declare-fun f (Int) Int)
(assert (forall ((i Int)) (< (f (+ i 1)) (f i))))
(assert (< (f 1) (f 0)))
(check-sat)
```



Type System

Unions, Intersections and Negations

```
function toInt (int|null x) -> (int r) :  
  //  
  if x is int:  
    // x: (int|null)&int == int  
    return x  
  else:  
    // x: (int|null)&!int == null  
    return x
```

- Variables automatically **retyped** after type tests
- **Intersections** and **negations** determine type on each branch

Recursive Types

```
type List is      null | {any data, List next}  
type IntList is null | {int data, IntList next}  
  
function length(List list) -> int:  
    if list is null:  
        return 0  
    else:  
        return 1 + length(list.next)
```

- Whiley supports true **recursive types**
- This is **necessary** for structural typing
- For example, IntList is a subtype of List

Empty Types

// What is this ??

```
type Invisible is Invisible | Invisible
```

// This is a list of infinite length ??

```
type InfList is { int data, InfList next }
```

```
function get(InfList l) -> (int d, InfList r):  
    return l.data, l.next
```

- Compiler should spot **empty** types and report them ...
- ... though does a **poor job** at the moment

Reference Lifetimes

```
method createPoint() -> &Point:  
    &this:Point local = this:new Point(0,0)  
    return local
```

- **Reference** types are of form `&T`
- **Lifetimes** extend syntax to `&l : T`
- Idea for reference lifetimes borrowed from **Rust!**

Conclusions

- Used for **teaching** SWEN224 “Software Correctness” since 2014
- Used for an **MSc** and a **PhD** so far, and 10+ **final** year projects
- **Preliminary investigation** in verifying challenge examples (VSCOMP’10,COST’11)
- Mark Utting working on using **Boogie** as alternative verification back-end
- ... but **lots** more work to do!!

`http://whiley.org`

@WhileyDave

References

- **A Mechanical Soundness Proof for Subtyping over Recursive Types.** T. Jones and D. J. Pearce. In FTfJP'16
- **Designing a Verifying Compiler: Lessons Learned from Developing Whiley.** D. J. Pearce and L. Groves. In *Science of Computer Programming*, 2015.
- **The Whiley Rewrite Language (WyRL).** D. J. Pearce. In SLE'15
- **Integer Range Analysis for Whiley on Embedded Systems.** D. J. Pearce. In SEUS'15
- **Reflections on Verifying Software with Whiley.** D. J. Pearce and L. Groves. In Proc FTSCS, 2013.
- **Whiley: a Platform for Research in Software Verification.** D. J. Pearce and L. Groves. In SLE'13.
- **A Calculus for Constraint-Based Flow Typing.** D. J. Pearce. In FTfJP'13.
- **Sound and Complete Flow Typing with Unions, Intersections and Negations,** D. J. Pearce. In VMCAI'13.
- **Implementing a Language with Flow-Sensitive and Structural Typing on the JVM.** D. J. Pearce and J. Noble. In BYTECODE'11.